



MASTG

- MASTG-TEST-0208: Insufficient Key Sizes
- MASTG-TEST-0212: Use of Hardcoded Cryptographic Keys in Code
- MASTG-TEST-0221: Broken Symmetric Encryption Algorithms
- MASTG-TEST-0232: Broken Symmetric Encryption Modes
- MASTG-TEST-0307: References to Asymmetric Key Pairs Used For Multiple Purposes
- MASTG-TEST-0308: Runtime Use of Asymmetric Key Pairs Used For Multiple Purposes
- MASTG-TEST-0309: References to Reused Initialization Vectors in

L2

ANDROID

MASWE-0012

TEST



MASTG-TEST-0307: References to Asymmetric Key Pairs Used For Multiple Purposes

Overview

According to section "5.2 Key Usage" of [NIST SP 800-57 part 1 revision 5](#), cryptographic keys should be assigned a specific purpose and used only for that purpose (e.g., encryption, integrity authentication, key wrapping, random bit generation, or digital signatures). For example, a key intended for encryption should not be used for signing.

On Android, asymmetric keys are commonly generated with `java.security.KeyPairGenerator` configured through `android.security.keystore.KeyGenParameterSpec`.

The `KeyGenParameterSpec.Builder` constructor has two arguments: the `keystoreAlias` and `purposes`, a bitmask of allowed operations documented in `android.security.keystore.KeyProperties`.

- `KeyProperties.PURPOSE_SIGN`
- `KeyProperties.PURPOSE_VERIFY`
- `KeyProperties.PURPOSE_ENCRYPT`
- `KeyProperties.PURPOSE_DECRYPT`
- `KeyProperties.PURPOSE_WRAP_KEY`

Steps

- Run static analysis ([Static Analysis on Android](#)) on the app and look for key generation code for asymmetric keys.

Observation

The output should contain a list of locations where asymmetric keys are created using `KeyGenParameterSpec.Builder` and the associated purposes.

Evaluation

The test case fails if you find any keys used for multiple roles (groups of purposes).

Using the output, ensure that each key pair is restricted to exactly **one** of the following roles:

- Encryption/Decryption (`PURPOSE_ENCRYPT` / `PURPOSE_DECRYPT`)
- Signing/Verification (`PURPOSE_SIGN` / `PURPOSE_VERIFY`)
- Key Wrapping (`PURPOSE_WRAP_KEY`)

When reverse engineering the app, you will find the previously mentioned purpose constants combined into a single integer value. For example, a purpose value of `15` combines all four purposes, which is not acceptable:

(`PURPOSE_ENCRYPT` = 1) | (`PURPOSE_DECRYPT` = 2) | (`PURPOSE_SIGN` = 4) | (`PURPOSE_VERIFY` = 8) = 15

Acceptable purpose combinations are:

- (`PURPOSE_ENCRYPT` = 1) = 1
- (`PURPOSE_DECRYPT` = 2) = 2
- (`PURPOSE_SIGN` = 4) = 4
- (`PURPOSE_VERIFY` = 8) = 8
- `PURPOSE_WRAP_KEY` = 32
- (`PURPOSE_ENCRYPT` = 1) | (`PURPOSE_DECRYPT` = 2) = 3
- (`PURPOSE_SIGN` = 4) | (`PURPOSE_VERIFY` = 8) = 12

Demos

MASTG-DEMO-0071: References to Asymmetric Key Pairs Used For Multiple Purposes with Semgrep